

AIC SFP Insertion Dataset

Technical Reference for Teammates

AI for Industry Challenge | UR5e Autonomous Cable Insertion
Recorded April 2026 | LeRobot v3.0 format

1. Dataset Overview

This dataset contains autonomous SFP cable insertion demonstrations collected in Gazebo simulation using a UR5e arm on the AIC task board. Each episode records the robot attempting to insert an SFP plug into a specific port on a NIC card. Data is stored in LeRobot v3.0 format (Parquet + MP4 video) and is directly compatible with the LeRobot training framework.

Available Batches

Batch	Episodes	Frames	~Size	Notes
batch_001	25	12 598	300 MB	Baseline; 5 NIC mounts targeted
batch_002	25	12 469	300 MB	Scene-randomised
batch_003	25	15 542	300 MB	Longer episodes (more descend ticks)
batch_004	25	12 399	300 MB	Scene-randomised
batch_005	25	12 396	300 MB	Scene-randomised
batch_006	25	12 476	300 MB	Scene-randomised
batch_007	25	12 415	300 MB	Scene-randomised
batch_008	2	983	300 MB	Partial (interrupted)
sreehari_batch	290	160 372	300 MB	Sreehari collected batch

Total usable: 7 complete batches = 175 episodes, ~87 000 frames (~7 hours of robot time). sreehari_batch adds 290 more episodes (~160k frames) for a combined total of ~465 episodes.

2. What Is in Each Episode

Each episode captures a single insertion attempt from pre-approach to hold (or timeout). Every frame is timestamped at 20 Hz and contains:

- observation.state -- 32-dim proprioceptive state vector (see Section 3)
- action -- 7-dim delta-pose command that was sent to the robot
- observation.images.* -- three camera views as AV1-encoded MP4 video
- task_index -- index into tasks.parquet (identifies which port/card)
- timestamp, frame_index, episode_index, index -- bookkeeping scalars

Episode Metadata (meta/ folder)

File	Contents
info.json	Dataset-level stats: total episodes, frames, fps, feature schema

episodes.jsonl	Per-episode: outcome, duration, AIC score, phase ticks, plug-port distance
tasks.parquet	Unique task strings (language labels) mapped to task_index integer
stats.json	Per-feature mean/std/min/max -- used for normalisation at training time

Task strings in batch_001 (5 unique tasks)

```
Insert sfp cable into nic_card_mount_0 sfp_port_0 (task_index=1)
Insert sfp cable into nic_card_mount_1 sfp_port_0 (task_index=0)
Insert sfp cable into nic_card_mount_2 sfp_port_0 (task_index=4)
Insert sfp cable into nic_card_mount_3 sfp_port_0 (task_index=2)
Insert sfp cable into nic_card_mount_4 sfp_port_0 (task_index=3)
```

This string is the ONLY signal that tells the model WHICH port to target at inference time. The port TF (ground-truth pose) is not available during evaluation per the AIC challenge rules. A standard ACT model ignores this string -- see Section 5 for policy choices.

3. Observation Space -- 32-dim state vector

The `observation.state` tensor has 32 float32 dimensions. Named slice constants are defined in `zss_collection/obs.py` and should be used in model architectures instead of hardcoded integers:

```
from zss_collection.obs import TCP_POSE_SLICE, TCP_VEL_SLICE, TCP_ERR_SLICE
from zss_collection.obs import JOINT_SLICE, FORCE_SLICE, TORQUE_SLICE

proprio    = obs[TCP_POSE_SLICE]    # dims 0-6   -> (7,)
velocity   = obs[TCP_VEL_SLICE]    # dims 7-12 -> (6,)
error      = obs[TCP_ERR_SLICE]    # dims 13-18 -> (6,)
joints     = obs[JOINT_SLICE]       # dims 19-25 -> (7,)
force      = obs[FORCE_SLICE]       # dims 26-28 -> (3,)
torque     = obs[TORQUE_SLICE]      # dims 29-31 -> (3,)
```

Slice	Dims	Field	Units	Notes
TCP_POSE_SLICE	0-6	tcp_pose (xyz + qxyzw)	m / --	TCP in base_link frame
TCP_VEL_SLICE	7-12	tcp_velocity (linear+angular)	m/s, rad/s	Cartesian velocity
TCP_ERR_SLICE	13-18	tcp_error (xyz + rxryrz)	m, rad	Controller tracking error
JOINT_SLICE	19-25	joint_positions (j0-j5 + j6)	rad	j6 always 0.0 (gripper fixed)
FORCE_SLICE	26-28	compensated_force (xyz)	N	FTS tared before each trial
TORQUE_SLICE	29-31	compensated_torque (xyz)	N.m	FTS tared before each trial

Why this observation design?

- TCP pose (absolute) gives global context about where the hand is relative to the base -- important for the approach phase.
- TCP velocity and error together tell the model how well the controller is tracking and the current motion speed. Insertion phases are distinguishable purely from the magnitude of these two signals.
- Joint positions are redundant kinematic context; they help the model resolve Cartesian ambiguity near singularities.
- Force/torque (compensated) are the key contact signals. During descent, Z-force rises to ~10-20 N at insertion. The model can learn to detect insertion completion from this signal alone.
- j6 (dim 25) is always 0.0 -- gripper stays fixed throughout collection. Acts as a constant padding slot.
- IMPORTANT: The reference `RunACT.py` policy was built against a 26-dim state (no F/T). Any model trained on `zss_collection` data must use `obs_dim=32`. Do not feed this data to the old 26-dim ACT without rebuilding it.

4. Action Space -- 7-dim delta pose in TCP frame

Dims	Field	Units	Frame
0-2	delta_position (dx, dy, dz)	m	Current TCP frame
3-6	delta_orientation (dqx, dqy, dqz, dqw)	--	Current TCP frame

Why delta pose in TCP frame?

- Board-position-invariant: the same insertion motion at two different board locations produces nearly identical action sequences. Demonstrations from different scenes are directly comparable with no post-processing.
- TCP-frame expression: the delta is in the current end-effector frame (not base_link), so the policy does not need to learn a separate motion profile for each arm configuration.

- Small magnitudes: deltas are ≤ 1 mm/tick laterally and ≤ 0.8 mm/tick downward. Normalisation is stable and training loss is well-conditioned.
- Stiffness is NOT in the action. It is a fixed controller parameter (ControllerParams.stiffness). This prevents the model from varying impedance, which would complicate force-based insertion detection.
- At inference: send the predicted delta as MODE_POSITION in the gripper/tcp frame. The Cartesian impedance controller interprets it as an incremental move from the current pose.

Inference adapter (pseudocode)

```
# In your policy node (zss_policy or custom wrapper):
delta_pos, delta_quat = pred_action[:3], pred_action[3:]

cmd = ControllerCommand(
    mode=MODE_POSITION,
    frame='gripper/tcp',    # <-- TCP frame, NOT base_link
    position=delta_pos,
    orientation=delta_quat,
)
```

5. Choosing a Training Policy

The dataset includes multi-task episodes (different NIC cards per episode). Your policy choice depends on whether you need the model to distinguish targets:

Policy	Task string used	Best for	Training flag
act	No	Single-target batches (one port, one card)	--policy.type=act
diffusion	No	Single-target batches	--policy.type=diffusion
smolvla	Yes	Multi-port / multi-card batches	--policy.type=smolvla
pi0	Yes	Multi-port / multi-card + richer images	--policy.type=pi0

ACT and diffusion policies completely ignore the task string.

If your batch targets multiple ports or NIC cards, the model will average over all targets and produce confused motions that point between ports.

Use smolvla / pi0 for multi-task, OR split into per-target single-task batches.

6. Training Commands

ACT -- single-task batch

```
cd ~/ws_aic/src/aic
pixi run lerobot-train \
  --dataset.repo_id=local/batch_001 \
  --dataset.root=/home/abinav2695/datasets_new/batch_001 \
  --policy.type=act \
  --policy.chunk_size=100 \
  --policy.n_action_steps=100 \
  --output_dir=outputs/train/act_batch_001 \
  --batch_size=16 \
  --steps=100000 \
  --log_freq=250 \
  --save_freq=10000 \
  --policy.device=cuda \
  --seed=42
```

Hyperparameter	Value	Rationale
chunk_size	100	5 s action window at 20 Hz -- covers the full CORRECT+DESCEND phases
n_action_steps	100	Execute full chunk before re-querying; avoids compounding re-query latency
batch_size	16	Stable for ~12 k frames; increase to 32 if >16 GB VRAM available
steps	100k	ACT needs many gradient steps on small (<200 episode) datasets

SmolVLA -- multi-task / multi-card batch

```
pixi run lerobot-train \
  --dataset.repo_id=local/batch_001 \
  --dataset.root=/home/abinav2695/datasets_new/batch_001 \
  --policy.type=smolvla \
```

```
--output_dir=outputs/train/smolvla_batch_001 \
--batch_size=8 \
--steps=100000 \
--policy.device=cuda
```

SmolVLA reads the per-episode task string and conditions every action prediction on it. The model learns which port to target purely from the language instruction (e.g. 'Insert sfp cable into nic_card_mount_2 sfp_port_0').

Combining multiple batches

```
# LeRobot supports merging datasets at training time:
pixi run lerobot-train \
  --dataset.repo_id=local/combined \
  --dataset.root=/home/abinav2695/datasets_new \
  --dataset.episodes='batch_001+batch_002+batch_003' \
  --policy.type=smolvla \
  --output_dir=outputs/train/smolvla_combined \
  ...
```

7. Using a Trained Checkpoint

After training, checkpoints are saved under output_dir:

```
outputs/train/act_batch_001/
  checkpoints/
    100000/
      config.json                # policy config
      model.safetensors          # weights
      policy_preprocessor_*_normalizer.safetensors # normalisation stats
```

In your policy node, replace `snapshot_download()` with the local checkpoint path:

```
from pathlib import Path

policy_path = Path(
    '~/ws_aic/src/aic/outputs/train/act_batch_001/checkpoints/100000'
).expanduser()

# Pass policy_path wherever snapshot_download(...) was used
```

The reference `aic_example_policies/ros/RunACT.py` uses a 7-dim ABSOLUTE pose + stiffness action expressed in base_link frame.

`zss_collection` records a 7-dim DELTA pose in the current TCP frame.

You MUST send predicted deltas as `MODE_POSITION` in the gripper/tcp frame.

Do NOT pass them as absolute targets in base_link -- the robot will jump to wrong positions.

8. Episode Insertion Phase Sequence

```
PRE_APPROACH -> APPROACH -> CORRECT -> DESCEND -> HOLD -> DONE
```

Phase	Ticks	Duration	What happens
PRE_APPROACH	~70	~3.5 s	Moves to overview height -- all NIC mounts visible in all cameras

AIC SFP Insertion Dataset -- Internal Technical Reference

APPROACH	~100	~5 s	Ramps TCP toward the target port at 20 cm above its surface
CORRECT	<=200	<=10 s	XY integrator loop reduces lateral error to <20 mm of port axis
DESCEND	var	var	Steps 0.8 mm down per tick; halts at insertion event or 40 N
HOLD	~100	~5 s	Maintains inserted pose; records confirmation frames

All phases are recorded in a single contiguous episode. The policy must learn the full sequence. `chunk_size=100` (5 s) is chosen to span the CORRECT+DESCEND phases where critical motions happen. The PRE_APPROACH phase is important: it teaches the policy to visually locate the target NIC card before moving toward it.

9. Quick-Reference Summary

Property	Value
Robot	UR5e (AIC configuration)
Task	SFP cable insertion into NIC card ports
Format	LeRobot v3.0 (Parquet + MP4 AV1)
Recording rate	20 Hz
obs.state dim	32 float32
action dim	7 float32 (delta pose in TCP frame)
Camera resolution	256x288x3 per camera (AV1 codec, 20 fps)
Cameras	left_camera, center_camera, right_camera
Max episode length	3600 ticks = 3 minutes hard timeout
Descent force limit	40 N (AIC penalty: >20 N sustained >1 s)
Usable batches	7 complete batches = 175 episodes (~87 000 frames)
sreehari_batch	290 episodes, 160 372 frames
Dataset root	/home/abinav2695/datasets_new/
Batch config root	~/ws_aic/src/aic/zss_data/configs/
Training runner	pixi run lerobot-train (from ~/ws_aic/src/aic/)

Key files

File	Purpose
zss_collection/zss_collection/obs.py	Named slice constants for obs.state (32-dim)
zss_collection/zss_collection/action.py	Action builder; delta pose design rationale
zss_collection/zss_collection/config.py	All tunable parameters (InsertionConfig etc.)
zss_collection/README.md	Full collection pipeline documentation
zss_data/configs/<batch>/launch_manifest.json	Per-trial Gazebo launch configuration
<batch>/meta/info.json	Feature schema and per-dataset statistics
<batch>/meta/tasks.parquet	Task string <-> task_index mapping
<batch>/meta/episodes.jsonl	Per-episode outcome, AIC score, phase ticks

Next steps / checklist

- To collect more data: use zss_collection/scripts/collect_orch.sh + collection_node (see zss_collection/README.md)
- For single-target training: run ACT with chunk_size=100, steps=100000 -- see Section 6
- For multi-card / multi-port targeting: use smolvla or pi0 (ACT/diffusion ignores the task string)
- For obs_dim compatibility: zss_collection = 32-dim. The old RunACT.py expects 26-dim -- do not mix
- For inference: send delta action as MODE_POSITION in gripper/tcp frame (not absolute in base_link)
- For force-safe deployment: monitor compensated_force.z; halt if >40 N (policy should learn this automatically)